The toy harness is a worker

MacBeth, Kodamai

11 October 2026

Prepared for: Robin Langer. Note 5, continuing Note 4 (harness-is-a-worker).
Corresponds to work-in-progress commit `commit 1985009`.

Abstract

Note 3 (workers-state-via-tensor) proved that a worker of state S from an input interface X to an output interface Y is exactly a container morphism $w : \Delta S \otimes X \to Y$, where $\Delta S = S \triangleright (\lambda_.\, S)$ is the state object. Robin asks how his *toy harness* — the fifteen-line agent loop of `toy-harness.pdf` — is such a worker. This note is the dictionary. There is no new theorem: it is an identification of a concrete system with the proved worker construction (the state-object result, `proofs/registry/state-object-delta.json`, is `proved` and Lean-verified).

1 Recap (Note 3)

A worker is a container morphism

$$w : \Delta S \otimes X \longrightarrow Y, \qquad X = X_0 \triangleright X_1, \quad Y = Y_0 \triangleright Y_1.$$

By Theorem 1 of Note 3 it unfolds into a forward map and a backward map:

$$\mathrm{fwd} : S \times X_0 \longrightarrow Y_0, \qquad \mathrm{bwd} : \prod_{(s,x_0)} Y_1\big(\mathrm{fwd}(s,x_0)\big) \longrightarrow S \times X_1(x_0).$$

By the universal property of $\times$, the backward map bwd splits into two components:

$$\mathrm{bwdS} : \prod_{(s,x_0)} Y_1\big(\mathrm{fwd}(s,x_0)\big) \longrightarrow S \qquad \text{(writeback of a new state)},$$

$$\mathrm{bwdX} : \prod_{(s,x_0)} Y_1\big(\mathrm{fwd}(s,x_0)\big) \longrightarrow X_1(x_0) \qquad \text{(ordinary backward map on } X\text{)}.$$

Slogan (Note 3). Control flows down (delegate a shape to Y), data flows back up (amalgamate Y's returned position into a new state and an X-answer). States multiply along $\otimes$:

$$\Delta S \otimes \Delta T = \Delta(S \times T), \qquad \Delta 1 = y \qquad \text{(Lemma 2 of Note 3)}.$$

2 The toy harness, as data

From `toy-harness.pdf`.

- The transcript `messages` is the harness's *only* state. So set

$$S := \text{the set of transcripts}$$

  — finite lists of messages, each a role $\in \{\texttt{user}, \texttt{assistant}\}$ with a list of content blocks $\in \{\texttt{text}, \texttt{tool_use}, \texttt{tool_result}, \texttt{thinking}\}$.

- The system prompt is a fixed side-channel that "never grows" and is resent every call. In worker terms it is *not* part of the state object: it is a constant, i.e. it contributes the tensor unit $\Delta 1 = y$ (tensoring by it changes nothing).

Remark 1. A non-growing side-channel is $\Delta 1$.

One pass of the `for`-loop is the fundamental transition. Model it as a worker

$$\text{step} : \Delta S \otimes X \longrightarrow Y$$

with the following data.

- Y is the delegated-to service = one model call together with the dispatch of its tool requests.
  - $Y_0 :=$ the set of API requests actually sent: the triple $(\texttt{system}, \texttt{tools}, \texttt{messages})$, but `system` and `tools` are fixed, so effectively the current transcript to `POST`.
  - $Y_1(y_0) :=$ the set of service replies over that request = a `response.content` (assistant blocks) together with, for each `tool_use` block, the `tool_result` returned by dispatch.
- X is the harness's *own* outward interface to its caller:
  - $X_0 :=$ pending tasks (a task string);
  - $X_1(x_0) :=$ answers (the returned text string).

The maps.

- $\text{fwd}(s, x_0)$: if s is empty, seed it with the user message $\{\texttt{role:user},\ \texttt{content} : \texttt{task} = x_0\}$; send the current transcript as the request y_0. (That is, fwd builds the request to `POST` from state + task.)
- On receiving a reply $r \in Y_1(\text{fwd}(s, x_0))$ — i.e. the assistant content plus the dispatched `tool_results` —

$$\text{bwdS}(r) = s +\!\!+ [\texttt{assistant:}\ \ \texttt{response.content}] +\!\!+ [\texttt{user:}\ \ \texttt{tool_results}]$$
$$\text{bwdX}(r) = \texttt{"".join(text blocks)}.$$

  Here $\text{bwdS}(r)$ is exactly the two `messages.append(...)` calls of the loop body (the new transcript). The map bwdX is defined precisely on those replies r whose `stop_reason` $\neq$ `"tool_use"` (the halt branch `return`); on `tool_use` replies bwdX is undefined-as-a-final-answer and the loop continues.

Proposition 1 (the loop body is a worker)**.** *The body of* ***`run_agent`*** *is a container morphism* $\text{step} : \Delta S \otimes X \to Y$ *with the* fwd, bwdS, bwdX *above. The two* ***`messages.append`*** *lines* are bwdS *(state writeback); the* ***`return`*** *line* is bwdX *(answer up); the* ***`client.messages.create(...)`*** *call followed by* ***`dispatch`*** is *the delegation* fwd *into* Y *and the receipt of a* Y_1*-position.*

3 The loop is coKleisli iteration

The full `run_agent` runs `step` repeatedly, threading the new transcript bwdS(r) back in as the next state, halting the first time a reply has `stop_reason` $\neq$ `tool_use` (then bwdX fires).

Proposition 2. *`run_agent` is the coKleisli iteration of the worker `step` for the graded comonad $p \mapsto \Delta S \otimes p$. Because state multiplies along $\otimes$ ($\Delta S \otimes \Delta T = \Delta(S \times T)$, Lemma 2 of Note 3), the accumulated state after k turns is the* concatenated *transcript — the growth of `messages` is exactly the state-multiplication of composed workers. (The `MAX_TURNS` bound is a cap on the iteration; it is not part of the worker's type.)*

4 `spawn_agent` is worker composition (nesting)

`run_agent` calls `dispatch` calls `run_agent`: the child is itself a worker

$$\text{child} : \Delta S_{\text{child}} \otimes (\texttt{task}) \longrightarrow (\texttt{string})$$

whose *entire* state S_{child} is a fresh transcript seeded with only the task string. When the child halts, one string crosses back and becomes *one* `tool_result` in the parent's transcript.

Remark 2. This is composition/nesting of workers: the child's store S_{child} is independent of and invisible to the parent — the parent amalgamates only the child's single bwdX answer (one string) as one Y_1-position. The child's state is existentially hidden, exactly the amalgamation of $\otimes$-composition. "Control down (delegate the task string), data up (one string)" is the container-morphism slogan read on `spawn_agent`.

5 The gate is a smart constructor

`check_permission` sits between the ask and the execution, with `ALLOW`/`DENY` lists; a denial returns an ordinary `tool_result` with an error flag, indistinguishable from a broken tool from inside.

Remark 3. The gate is a sub-object inclusion on the delegated shapes Y_0 (the `ALLOW` sub-container of possible tool calls) composed into fwd; a denial is still a genuine Y_1 position (the error `tool_result`), so the gate does not change the worker's *type*, only its forward map — "a denied command and a broken tool look the same from inside" is precisely the statement that both are ordinary positions of Y_1. This instantiates the seed's "trust boundary = smart constructor."

6 What is NOT state (honesty)

The bash working directory persists across calls but is external world-state, not part of S (the harness does not carry it in `messages`); model it as an ambient effect, not the state object. The `depth` and `turn` counters bound the recursion/iteration and are parameters, not state.

Punchline. The transcript is the store, one turn is fwd-then-writeback, the loop is coKleisli iteration, spawn is nesting, and the permission gate is a smart constructor — the fifteen-line harness is a worker on the nose.